

Q.NO: 2

Question description

There is a bit string consisting of n bits. Then, there are some changes that invert one given bit. Your task is to report, after each change, the length of the longest substring whose each bit is the same.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq m \leq 2 \cdot 10^5$
- $1 \leq x_i \leq n$

Input Format

The first input line has a bit string consisting of n bits. The bits are numbered $1, 2, \dots, n$.
The next line contains an integer m : the number of changes.
The last line contains m integers $x_1, x_2, \dots, x_m$ describing the changes.

Output Format

After each change, print the length of the longest substring whose each bit is the same.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

001011
3
3 2 5

EXPECTED OUTPUT

4 2 3

Test Case 2

INPUT (STDIN)

101010
4
1 3 2 6

EXPECTED OUTPUT

2 4 2 2

▼ Mandatory Test Cases

Test Case 1

Code:

```
#include <stdio.h>
#include <string.h>

#define MAX 200005

char s[MAX];

int len0[4 * MAX];
int len1[4 * MAX];
```

```
int left0[4 * MAX];
int left1[4 * MAX];
int right0[4 * MAX];
int right1[4 * MAX];

void pull(int k, int l, int r)
{
    int mid = (l + r) / 2;
    int left = k * 2;
    int right = k * 2 + 1;

    len0[k] = len0[left] > len0[right] ? len0[left] : len0[right];
    len1[k] = len1[left] > len1[right] ? len1[left] : len1[right];

    left0[k] = left0[left];
    left1[k] = left1[left];

    right0[k] = right0[right];
    right1[k] = right1[right];

    if(left0[left] == mid - l + 1)
        left0[k] += left0[right];

    if(left1[left] == mid - l + 1)
        left1[k] += left1[right];

    if(right0[right] == r - mid)
        right0[k] += right0[left];

    if(right1[right] == r - mid)
        right1[k] += right1[left];

    int cross0 = right0[left] + left0[right];
    int cross1 = right1[left] + left1[right];

    if(cross0 > len0[k])
        len0[k] = cross0;

    if(cross1 > len1[k])
        len1[k] = cross1;
}

void build(int k, int l, int r)
{
    if(l == r)
    {
        if(s[l] == '0')
        {
            len0[k] = 1;
            left0[k] = 1;
            right0[k] = 1;
        }
    }
}
```

```
len1[k] = 0;
left1[k] = 0;
right1[k] = 0;
}
else
{
len1[k] = 1;
left1[k] = 1;
right1[k] = 1;

len0[k] = 0;
left0[k] = 0;
right0[k] = 0;
}

return;
}

int mid = (l + r) / 2;

build(k * 2, l, mid);
build(k * 2 + 1, mid + 1, r);

pull(k, l, r);
}

void update(int k, int l, int r, int pos)
{
if(l == r)
{
if(s[l] == '0')
{
s[l] = '1';

len0[k] = 0;
left0[k] = 0;
right0[k] = 0;

len1[k] = 1;
left1[k] = 1;
right1[k] = 1;
}
else
{
s[l] = '0';

len1[k] = 0;
left1[k] = 0;
right1[k] = 0;

len0[k] = 1;
```

```
left0[k] = 1;
right0[k] = 1;
}

return;
}

int mid = (l + r) / 2;

if(pos <= mid)
    update(k * 2, l, mid, pos);
else
    update(k * 2 + 1, mid + 1, r, pos);

pull(k, l, r);
}

int main()
{
    int n, m;
    int x;

    printf("Input value:\n");

    scanf("%s", s + 1);

    n = strlen(s + 1);

    scanf("%d", &m);

    build(1, 1, n);

    printf("\nOutput:\n");

    for(int i = 0; i < m; i++)
    {
        scanf("%d", &x);

        update(1, 1, n, x);

        if(len0[1] > len1[1])
            printf("%d", len0[1]);
        else
            printf("%d", len1[1]);

        if(i < m - 1)
            printf(" ");
    }

    printf("\n");
```

```
return 0;  
}
```

Test_case 1:

Input value:

001011

3

3 2 5

Output:

4 2 3

Test_case 2:

Input value:

101010

4

1 3 2 6

Output:

2 4 2 2